

A06-Zombies

COMP301

Introduction

This assignment provides practice with asynchronous programming and synchronization in Java. You will implement two classes, **Base** and **Survivor**, which work together in a multithreaded survival simulation.

In this simulation, multiple Survivors will perform tasks asynchronously, interacting with a shared Base object to gather supplies and fortify defenses. Since multiple survivors will be accessing shared resources simultaneously, you must use synchronization techniques to ensure data consistency and avoid race conditions.

You will be provided with two interfaces, **IBase** and **ISurvivor**, which define the required functionality for the Base and Survivor classes. Your task is to implement these classes according to the specifications below.

Apprentice

Base

The **Base** class represents the survivors' home, where supplies are stored and tools are used. It must be thread-safe because multiple survivors may interact with it at the same time.

Interface: IBase

```
public interface IBase {  
    void addSupplies(int amount); // Adds supplies to the base  
    int getSupplyCount(); // Returns the current supply count  
    void useTool(String task); // Allows a survivor to use a tool  
}
```

Class Specification: Base

Your **Base** class must implement **IBase** and include the following functionality.

addSupplies(int amount)

The **addSupplies** method should be **synchronized** so that multiple survivors cannot modify the supply count at the same time. It takes an integer amount representing the number of supplies to add. If the amount is less than or equal to zero, the method should throw an **IllegalArgumentException**. Otherwise, it should increase the total supply count and print a message indicating how many supplies were added and the new total. You can get the name of the current caller by using **Thread.currentThread().getName()**.

Example output: Survivor-1 added 5 supplies. Total: 15

getSupplyCount()

Standard getter with a special case. The **getSupplyCount** method should also be **synchronized** since it reads the shared supply count. This method returns an integer representing the current total number of supplies.

`useTool(String task)`

The `useTool` method allows a survivor to perform a task using a tool. This method should not be synchronized, but instead, it should use a `Lock` to ensure that only one survivor can use a tool at a time. The method takes a string describing the task and should print a message indicating that the survivor is using a tool. To simulate the time required for tool use, the method should call `Thread.sleep(1000)`. After that, it should print a message indicating that the tool is no longer in use. The lock should always be released properly, even if an exception occurs.

Survivor

The `Survivor` class represents an individual survivor who performs actions asynchronously. Each survivor runs as a separate thread, performing tasks such as gathering supplies, fortifying the base, and resting.

Interface: `ISurvivor`

```
public interface ISurvivor extends Runnable {
    void stop(); // Stops the survivor thread
    @Override
    void run(); // Inherited from Runnable. Defines how the survivor executes
                tasks asynchronously
}
```

Class Specification: `Survivor`

Protected Methods

In addition to the exposed methods of the interface, the `Survivor` should be able to perform specific protected actions to help the community survive. The action methods should throw `InterruptedException`.

- `fortify()`
- `scavenge()`
- `rest()`
- `performAction()` (calls the other actions)

You may create other private methods as you deem necessary, but these must exist as protected (for testing purposes).

`performAction()`

The `performAction` method determines what the survivor does at any given moment. This method should randomly select one of three possible actions: scavenging for supplies, fortifying the base, or resting.

`fortify()`

The `fortify` method represents a survivor using a tool to strengthen the base. This method should call the `useTool` method of `Base` with the string `"fortification"`. It should also print a message indicating that the survivor is fortifying the base and take 2 seconds to complete.

`scavenge()`

The `scavenge` method represents a survivor searching for supplies. It should print a message indicating that the survivor is scavenging and then sleep for a random amount of time between one and four seconds to simulate the time taken to find supplies. After that, the survivor should add 2 supplies to the base.

`rest()`

The `rest` method represents a survivor taking a break. It should print a message indicating that the survivor is resting and then sleep for 2 seconds.

Public Methods

`run()` The `run` method defines the behavior of a survivor when it is running as a thread. The method should continuously execute until the survivor is stopped. In each iteration of its loop, the survivor should call `performAction` to randomly choose an action. Here is where you have to deal with the `InterruptedException`. If the survivor is interrupted, it must signal that the survivor needs to stop to ensure proper thread handling.

`stop()` The `stop` method provides a way to stop the survivor safely. Instead of terminating the thread forcefully, this method should set a `volatile boolean` flag to false, allowing the survivor's run loop to exit cleanly.

Pro Tips: Your implementation should ensure that Survivor runs as a separate thread by implementing `Runnable`. The survivor should check a volatile boolean flag to determine when to stop running. If the survivor is interrupted, it should exit the loop immediately. Synchronization must be used in `Base` to prevent race conditions when updating supplies. The `useTool` method should use a `Lock` instead of the `synchronized` keyword so that only one survivor can use a tool at a time. Randomization should be used to determine which action a survivor takes when calling `performAction`.

Enchanter

ZombieApocalypse Simulation

The `ZombieApocalypse` class is responsible for managing the entire survival simulation. It initializes survivors, runs the simulation for a set period, and then stops all survivor threads. This class provides structure for handling asynchronous survivor actions.

Methods Overview

- `startSimulation(int numSurvivors)`
- `simulateDayNightCycle(int milliseconds, IDayStrategy events)`
- `endSimulation()`
- `main()`

All methods should be public. To keep things simple, make everything static.

Instance Variables

```
protected static Base base;  
protected static Survivor[] survivors;  
protected static Thread[] survivorThreads;
```

```
public static void startSimulation(int numSurvivors)
```

Sets up the survival environment by initializing the base and creating survivor threads. Each survivor runs concurrently, performing actions asynchronously.

```
public static void simulateDayNightCycle(int milliseconds, IDayStrategy events)
```

Allows time to pass in the simulation before stopping. Execute the day's strategy using the base and handle `InterruptedException` properly.

```
public static void endSimulation()
```

Stops all survivors and ensures their threads terminate properly. Prints “Simulation ending...”, calls `stop()` on each survivor, joins threads, and prints “All survivors have stopped. Simulation over.”

```
public static void main(String[] args)
```

Entry point of the program. Calls `startSimulation()`, `simulateDayNightCycle()`, and `endSimulation()` to run a full simulation with five survivors and a given `QuietDayStrategy` day-night cycle.

Sorcerer

Zombie Attack!!!

Adding Zombie Attacks to the Base Class

To simulate a true zombie apocalypse, the `Base` class needs to be able to track whether the base is currently under attack by zombies. Add a private boolean `isUnderAttack` and three synchronized methods: `startAttack()`, `endAttack()`, and `isUnderAttack()`.

- `isUnderAttack()` returns whether the base is under attack.
- `startAttack()` sets the attack state to true and prints that zombies are approaching.
- `endAttack()` sets it back to false and prints that survivors repelled the zombies.

Adding Zombie Attacks to the Survivor Class

When the base is under attack, survivors should defend instead of performing other actions. Add a boolean flag `defending` and getter `isDefending()`. When defending, survivors print a defense message, sleep for 2 seconds, and set `defending` back to false afterward.

Adding Zombie Attacks to the ZombieApocalypse Class

Replace the `QuietDayStrategy` with a new `RandomZombieAttacksStrategy` to trigger zombie attacks during the simulation.

Adding a Zombie Attack Strategy

Implement a class `RandomZombieAttacksStrategy` that implements `IDayStrategy`. Divide the total simulation time into two attacks and three rest periods. Each attack lasts one-fifth of the total time, and the rest of the time is evenly split before, between, and after attacks. Call `startAttack()` and `endAttack()` appropriately.

Merlin

Make Your Own Day

Finally, create your own `DayStrategy` that determines when and how often zombies will attack. Extra Credit will be awarded based on complexity (up to 5 points).

Happy Hunting!